PRD: Personalized Learning Path Recommender for Online Education

Oct 22, 2026 · @rishu

Product requirements for an AI/ML recommender that reads each learner's quiz performance and engagement patterns and returns a personalized sequence of learning content. Numeric targets are proposals to confirm.

1. Problem and vision

Online courses give every learner the same linear path, so strong learners get bored and struggling learners fall behind without anyone noticing. Instructors see only final grades, too late to help.

The product tracks what each learner knows per topic, recommends the next best piece of content, adjusts quiz difficulty as the learner improves, and shows progress to learners and instructors. Recommendation strategies are swappable so the team can A/B test which one actually improves learning.

Vision: every learner always knows what to study next, and why.

2. Goals, non-goals and success metrics

The product succeeds if learners finish more content and learn more per hour than with a fixed path. Targets are proposed baselines to confirm against real data.

Goal	Metric	Proposed target
Better learning outcomes	Average gain between pre-test and post-test scores	+15% vs fixed path
Higher completion	Learners finishing a course within 8 weeks	+20% relative
Relevant recommendations	Recommendation acceptance rate (learner opens suggested item)	60% or more
Efficient assessment	Questions needed for a stable mastery estimate	30% fewer than static quizzes
Instructor visibility	Instructors who view the dashboard weekly	70% of active instructors

Goal	Metric	Proposed target
Responsiveness	Time to return a recommendation	under 500 ms at p95

Non-goals for v1: authoring course content, live video or tutoring, payments and certificates, native mobile apps, and free-text answer grading by large language models.

3. Users and personas

Three roles use the system; learners are the primary audience.

Persona	Who they are	Main need	Pain today
Learner (Asha)	Working adult or student taking self-paced courses	Know what to study next and see progress	Unsure where gaps are; wastes time on content she already knows
Instructor (Prof. Rao)	Teaches one or more courses with 50 to 500 learners	Spot struggling learners and weak topics early	Sees only final grades; cannot personalize at scale
Admin / Content manager (Meera)	Maintains the course catalog and platform settings	Tag content, tune strategies, run experiments	Manual curation; no way to compare approaches

4. User stories

Priority uses MoSCoW: Must, Should, Could. Each story maps to SRS requirements in the SRS traceability matrix.

ID	As a...	I want to...	So that...	Priority
US-01	Learner	take a short diagnostic quiz on joining a course	the system knows my starting level	Must
US-02	Learner	see my strength per topic	I know where to focus	Must
US-03	Learner	get a recommended next lesson with a one-line reason	I trust and follow the path	Must
US-04	Learner	get quizzes that get harder or easier as I answer	I stay challenged but not frustrated	Must

ID	As a...	I want to...	So that...	Priority
US-05	Learner	skip or dismiss a recommendation	the path respects my preferences	Should
US-06	Learner	track streaks, time spent and completion	I stay motivated	Should
US-07	Instructor	see class-wide topic mastery and at-risk learners	I can intervene early	Must
US-08	Instructor	drill into one learner's history	I can give targeted help	Should
US-09	Admin	tag content with topic, difficulty and prerequisites	the recommender can use it	Must
US-10	Admin	create an experiment that splits learners across strategies	I can measure which works better	Must
US-11	Admin	switch a strategy on or off without a deploy	I can roll back quickly	Should
US-12	Learner	export my progress report	I can share it	Could

5. Scope

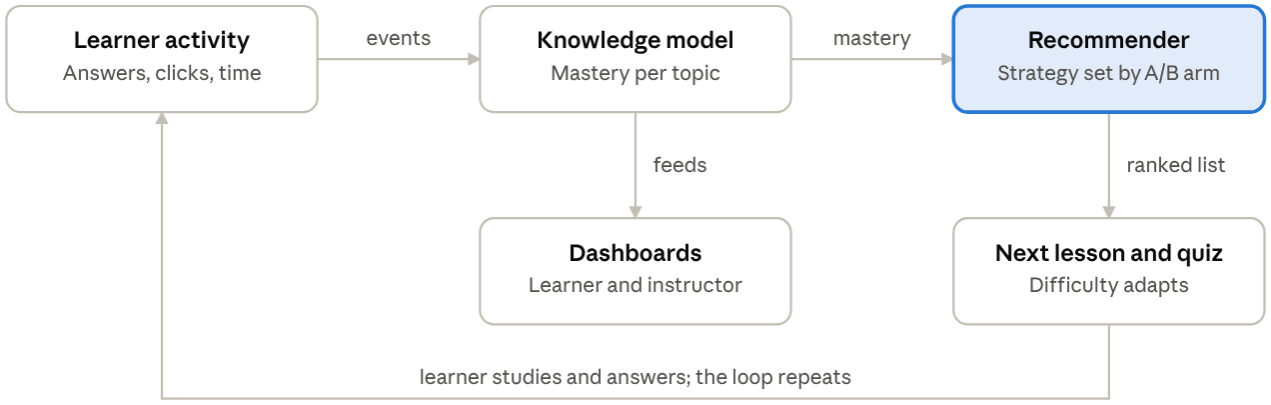
The MVP delivers the full loop (assess, model, recommend, adapt, show) with one simple recommender. Smarter models and experiments grow from there.

Capability	MVP	v1.1	Later
Knowledge model	Per-topic mastery score with confidence	Forgetting decay over time	Prerequisite-aware inference
Recommendation engine	Rule-based using mastery, prerequisites and difficulty	Content embeddings plus rules (hybrid)	Collaborative filtering from similar learners
Adaptive quizzing	Difficulty steps up or down by recent accuracy	Item-response-theory calibrated difficulty	Computerized adaptive testing
Learner dashboard	Mastery per topic, next steps, completion	Streaks and time-on-task	Goals and study planner

Capability	MVP	v1.1	Later
Instructor dashboard	Class mastery heatmap, at-risk list	Per-learner drill-down	Alerts and exports
A/B testing	Two strategies, random 50/50 split	Multiple arms, guardrail metrics	Bandit allocation
Admin	Content tagging, strategy toggle	Experiment console	Role-based org management

6. How it works

Every answer updates mastery, which steers the next lesson



learner loop · 5 steps, 1 feedback loop

Activity flows into the knowledge model; the recommender ranks content from mastery using whichever strategy the learner's A/B arm assigns; the next lesson and quiz generate new activity, closing the loop. Dashboards read the same mastery data.

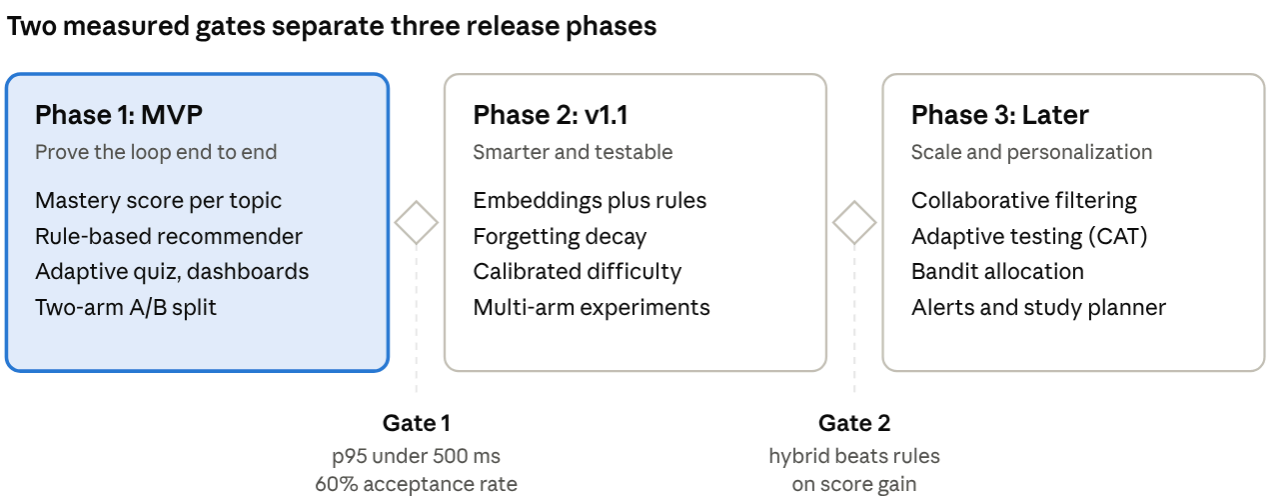
7. Experimentation plan

Every recommendation request goes through a strategy interface, so any strategy can be tested against another with no change to the learner experience.

- **Assignment:** learners are randomly and persistently assigned to one arm per experiment, so a learner never switches mid-test.
- **Candidate strategies:** (A) rule-based, (B) embedding plus rules, (C) collaborative filtering.
- **Primary metric:** post-test score gain; **secondary:** recommendation acceptance rate and completion within 8 weeks.

- **Guardrails:** dropout rate and quiz abandonment must not worsen by more than 5%; an experiment auto-pauses if breached.
- **Readout:** a significance test and a confidence interval per metric, shown to admins with sample size, run length and a minimum of 200 learners per arm.

8. Release plan



release roadmap · 3 phases, 2 gates

Phase durations are set once team size and data access are known. A phase starts only after the previous gate is met; gate thresholds reuse the targets in section 2.

9. Risks, assumptions and open questions

Risk	Likelihood	Impact	Mitigation
Cold start: new learners or new content have no history	High	Medium	Diagnostic quiz; fall back to rules and popularity
Sparse data makes collaborative filtering weak	High	Medium	Start rule-based; add collaborative filtering once data grows
Poorly tagged content gives bad recommendations	Medium	High	Required tagging fields; admin tag-coverage report
Learners distrust or ignore suggestions	Medium	Medium	Show a reason for each item; allow dismissing
Privacy of learner performance data	Medium	High	Consent, role-based access, data minimization

Risk	Likelihood	Impact	Mitigation
Small cohorts give inconclusive A/B tests	Medium	Medium	Minimum sample rule; run longer instead of peeking

Assumptions: the platform already has a course catalog with lessons and quizzes; content can be tagged by topic and difficulty; learners log in with an account.

Open questions:

- [ ] Is this a standalone web app or a plug-in to an existing LMS?
- [ ] Which subject area should the demo catalog cover (for example data science or programming)?
- [ ] Is a real dataset available, or should synthetic learner data be generated?
- [ ] Which tech stack is preferred (for example Python with FastAPI and React)?